Generic Banking Credit File example: build an Abstract Factory example around a credit file processing domain covering concepts like: Credit file parsers (retail vs corporate vs mortgage) Risk scoring engines (per credit type) Regulatory report generators (Basel III, IFRS 9, etc.) Approval workflow handlers

https://ms.sg.dev/deepsearch/9f3e2750-53bb-4b6f-beb9-0b44c6710646

Generic Banking Credit File example: build an Abstract Factory example around a credit file processing domain covering concepts like: Credit file parsers (retail vs corporate vs mortgage) Risk scoring engines (per credit type) Regulatory report generators (Basel III, IFRS 9, etc.) Approval workflow handlers

---

Banking Credit File example: build an Abstract Factory example in Java around a credit file processing domain covering concepts like: Credit file parsers (retail vs corporate vs mortgage) Risk scoring engines (per credit type) Regulatory report generators (Basel III, IFRS 9, etc.) Approval workflow handlers

---

Abstract Factory Pattern: Banking Credit File Processing

Design Overview

The Abstract Factory creates families of related objects — one factory per credit type (Retail, Corporate, Mortgage). Each factory produces four product types: Parser, Risk Scorer, Regulatory Reporter, and Approval Handler.

---

1. Product Interfaces

```
// --- Parser ---
public interface CreditFileParser {
    CreditFile parse(byte[] rawData);
}

// --- Risk Scorer ---
public interface RiskScoringEngine {
    RiskScore score(CreditFile creditFile);
}

// --- Regulatory Reporter ---
public interface RegulatoryReportGenerator {
    RegulatoryReport generate(CreditFile creditFile, RiskScore riskScore);
}

// --- Approval Handler ---
public interface ApprovalWorkflowHandler {
    ApprovalDecision handle(CreditFile creditFile, RiskScore riskScore);
}
```

---

2. Domain Model (Shared Data Objects)

```
public class CreditFile {
    private final String fileId;
    private final String applicantId;
    private final CreditType creditType;
    private final double requestedAmount;
    private final Map<String, Object> attributes; // flexible payload

    public CreditFile(String fileId, String applicantId,
                      CreditType creditType, double requestedAmount,
                      Map<String, Object> attributes) {
        this.fileId = fileId;
        this.applicantId = applicantId;
        this.creditType = creditType;
        this.requestedAmount = requestedAmount;
        this.attributes = Collections.unmodifiableMap(attributes);
    }

    // getters omitted for brevity
    public String getFileId()            { return fileId; }
    public String getApplicantId()       { return applicantId; }
    public CreditType getCreditType()    { return creditType; }
    public double getRequestedAmount()   { return requestedAmount; }
    public Map<String, Object> getAttributes() { return attributes; }
}
```

```
public enum CreditType { RETAIL, CORPORATE, MORTGAGE }

public record RiskScore(String fileId, double score, RiskBand band, String rationale) {
    public enum RiskBand { LOW, MEDIUM, HIGH, CRITICAL }
}

public record RegulatoryReport(String reportId, String framework,
                               String fileId, Map<String, Object> sections) {}

public record ApprovalDecision(String fileId, boolean approved,
                               String reason, String approverTier) {}
```

---

3. Abstract Factory Interface

```
/**
 * Abstract Factory: produces a consistent family of credit processing
 * components for a specific credit type.
 */
public interface CreditProcessingFactory {

    CreditFileParser         createParser();
    RiskScoringEngine        createRiskScoringEngine();
    RegulatoryReportGenerator createRegulatoryReportGenerator();
    ApprovalWorkflowHandler  createApprovalWorkflowHandler();

    CreditType supportedType();
}
```

---

4. Retail Credit Family

```
// --- Parser ---
public class RetailCreditFileParser implements CreditFileParser {

    @Override
    public CreditFile parse(byte[] rawData) {
        // Retail files: personal income, FICO score, DTI ratio
        String json = new String(rawData, StandardCharsets.UTF_8);
        Map<String, Object> attrs = new HashMap<>();
        attrs.put("fico_score",    extractField(json, "fico_score"));
        attrs.put("annual_income", extractField(json, "annual_income"));
        attrs.put("dti_ratio",     extractField(json, "dti_ratio"));
        attrs.put("employment_status", extractField(json, "employment_status"));
```

```
        return new CreditFile(
            UUID.randomUUID().toString(),
            extractField(json, "applicant_id").toString(),
            CreditType.RETAIL,
            Double.parseDouble(extractField(json, "requested_amount").toString()),
            attrs
        );
    }

    private Object extractField(String json, String field) {
        // simplified extraction — use Jackson/Gson in production
        return "[" + field + "_value]";
    }
}

// --- Risk Scorer ---
public class RetailRiskScoringEngine implements RiskScoringEngine {

    private static final double FICO_WEIGHT   = 0.50;
    private static final double DTI_WEIGHT    = 0.30;
    private static final double INCOME_WEIGHT = 0.20;

    @Override
    public RiskScore score(CreditFile creditFile) {
        Map<String, Object> attrs = creditFile.getAttributes();

        double ficoScore    = toDouble(attrs.get("fico_score"),    700);
        double dtiRatio     = toDouble(attrs.get("dti_ratio"),     0.40);
        double annualIncome = toDouble(attrs.get("annual_income"), 50_000);

        // Normalise: FICO 300-850, DTI 0-1, income scaled to 0-1 at $200k cap
        double ficoNorm   = (ficoScore - 300) / 550.0;
        double dtiNorm    = 1.0 - dtiRatio;                        // lower DTI = better
        double incomeNorm = Math.min(annualIncome / 200_000.0, 1.0);

        double compositeScore = (ficoNorm   * FICO_WEIGHT)
                              + (dtiNorm    * DTI_WEIGHT)
                              + (incomeNorm * INCOME_WEIGHT);

        RiskScore.RiskBand band = classifyBand(compositeScore);
        return new RiskScore(
            creditFile.getFileId(),
            compositeScore,
            band,
            String.format("Retail scoring: FICO=%.0f, DTI=%.2f, Income=%.0f → score=%.3f [%s]",
                          ficoScore, dtiRatio, annualIncome, compositeScore, band)
        );
    }

    private RiskScore.RiskBand classifyBand(double score) {
        if (score >= 0.75) return RiskScore.RiskBand.LOW;
        if (score >= 0.55) return RiskScore.RiskBand.MEDIUM;
        if (score >= 0.35) return RiskScore.RiskBand.HIGH;
```

```
        return RiskScore.RiskBand.CRITICAL;
    }

    private double toDouble(Object val, double fallback) {
        if (val == null) return fallback;
        try { return Double.parseDouble(val.toString()); }
        catch (NumberFormatException e) { return fallback; }
    }
}

// --- Regulatory Reporter (Basel III + IFRS 9 for retail) ---
public class RetailRegulatoryReportGenerator implements RegulatoryReportGenerator {

    @Override
    public RegulatoryReport generate(CreditFile creditFile, RiskScore riskScore) {
        Map<String, Object> sections = new LinkedHashMap<>();

        // IFRS 9 ECL staging for retail
        sections.put("ifrs9_stage",          resolveIfrs9Stage(riskScore));
        sections.put("pd_estimate",          estimatePD(riskScore.score()));
        sections.put("lgd_estimate",         0.45);   // retail unsecured standard
        sections.put("ead",                  creditFile.getRequestedAmount());
        sections.put("ecl",                  computeECL(riskScore, creditFile));

        // Basel III capital requirement (standardised approach)
        sections.put("risk_weight_percent",  resolveRiskWeight(riskScore));
        sections.put("rwa",                  computeRWA(creditFile, riskScore));
        sections.put("capital_requirement",  computeCapitalReq(creditFile, riskScore));

        return new RegulatoryReport(
            "REG-RETAIL-" + creditFile.getFileId(),
            "IFRS9 / Basel III",
            creditFile.getFileId(),
            sections
        );
    }

    private int resolveIfrs9Stage(RiskScore rs) {
        return switch (rs.band()) {
            case LOW      -> 1;
            case MEDIUM   -> 2;
            case HIGH, CRITICAL -> 3;
        };
    }

    private double estimatePD(double score) {
        // Logistic-style mapping: higher score → lower PD
        return Math.max(0.001, 1.0 - score);
    }

    private double computeECL(RiskScore rs, CreditFile cf) {
        double pd  = estimatePD(rs.score());
        double lgd = 0.45;
```

```
        double ead = cf.getRequestedAmount();
        return pd * lgd * ead;
    }

    private double resolveRiskWeight(RiskScore rs) {
        return switch (rs.band()) {
            case LOW      -> 0.75;   // retail IRB
            case MEDIUM   -> 1.00;
            case HIGH     -> 1.50;
            case CRITICAL -> 2.00;
        };
    }

    private double computeRWA(CreditFile cf, RiskScore rs) {
        return cf.getRequestedAmount() * resolveRiskWeight(rs);
    }

    private double computeCapitalReq(CreditFile cf, RiskScore rs) {
        return computeRWA(cf, rs) * 0.08;  // Basel III 8% minimum
    }
}

// --- Approval Handler ---
public class RetailApprovalWorkflowHandler implements ApprovalWorkflowHandler {

    private static final double RETAIL_AUTO_APPROVE_LIMIT = 50_000;
    private static final double RETAIL_AUTO_REJECT_THRESHOLD = 0.35;

    @Override
    public ApprovalDecision handle(CreditFile creditFile, RiskScore riskScore) {
        double amount = creditFile.getRequestedAmount();
        double score  = riskScore.score();

        if (score < RETAIL_AUTO_REJECT_THRESHOLD) {
            return new ApprovalDecision(creditFile.getFileId(), false,
                "Risk score below retail minimum threshold (" + score + ")",
                "AUTOMATED_RULES_ENGINE");
        }

        if (score >= 0.75 && amount <= RETAIL_AUTO_APPROVE_LIMIT) {
            return new ApprovalDecision(creditFile.getFileId(), true,
                "Auto-approved: low risk, within retail self-serve limit",
                "AUTOMATED_RULES_ENGINE");
        }

        if (score >= 0.55) {
            return new ApprovalDecision(creditFile.getFileId(), true,
                "Approved: medium risk — standard retail underwriter review",
                "RETAIL_UNDERWRITER_L1");
        }

        return new ApprovalDecision(creditFile.getFileId(), false,
            "Declined: high risk score — requires senior credit review",
```

```
            "RETAIL_CREDIT_COMMITTEE");
    }
}

// --- Factory ---
public class RetailCreditProcessingFactory implements CreditProcessingFactory {

    @Override public CreditFileParser          createParser()                 { return new RetailCreditFileParse
    @Override public RiskScoringEngine         createRiskScoringEngine()      { return new RetailRiskScoringEngi
    @Override public RegulatoryReportGenerator createRegulatoryReportGenerator(){ return new RetailRegulatoryRepo
    @Override public ApprovalWorkflowHandler   createApprovalWorkflowHandler() { return new RetailApprovalWorkflo
    @Override public CreditType                supportedType()                { return CreditType.RETAIL; }
}
```

---

5. Corporate Credit Family

```
// --- Parser ---
public class CorporateCreditFileParser implements CreditFileParser {

    @Override
    public CreditFile parse(byte[] rawData) {
        // Corporate: financials, credit ratings, covenant data
        String json = new String(rawData, StandardCharsets.UTF_8);
        Map<String, Object> attrs = new HashMap<>();
        attrs.put("sp_rating",          extractField(json, "sp_rating"));
        attrs.put("ebitda",             extractField(json, "ebitda"));
        attrs.put("debt_to_ebitda",     extractField(json, "debt_to_ebitda"));
        attrs.put("interest_coverage",  extractField(json, "interest_coverage"));
        attrs.put("industry_sector",    extractField(json, "industry_sector"));
        attrs.put("years_in_operation", extractField(json, "years_in_operation"));

        return new CreditFile(
            UUID.randomUUID().toString(),
            extractField(json, "company_id").toString(),
            CreditType.CORPORATE,
            Double.parseDouble(extractField(json, "facility_amount").toString()),
            attrs
        );
    }

    private Object extractField(String json, String field) { return "[" + field + "_value]"; }
}

// --- Risk Scorer ---
public class CorporateRiskScoringEngine implements RiskScoringEngine {

    // Internal rating scale mapped from S&P equivalents
    private static final Map<String, Double> RATING_SCORES = Map.of(
```

```
        "AAA", 1.00, "AA", 0.90, "A", 0.80,
        "BBB", 0.65, "BB", 0.50, "B", 0.35,
        "CCC", 0.20, "D",  0.00
    );

    @Override
    public RiskScore score(CreditFile creditFile) {
        Map<String, Object> attrs = creditFile.getAttributes();

        double ratingScore   = RATING_SCORES.getOrDefault(
                                   attrs.getOrDefault("sp_rating","BBB").toString(), 0.65);
        double debtEbitda    = toDouble(attrs.get("debt_to_ebitda"), 4.0);
        double coverage      = toDouble(attrs.get("interest_coverage"), 3.0);

        double leverageScore = Math.max(0, 1.0 - (debtEbitda / 10.0));
        double coverageScore = Math.min(coverage / 8.0, 1.0);

        double composite = (ratingScore   * 0.50)
                         + (leverageScore * 0.30)
                         + (coverageScore * 0.20);

        RiskScore.RiskBand band = classifyBand(composite);
        return new RiskScore(
            creditFile.getFileId(), composite, band,
            String.format("Corporate scoring: Rating=%s, D/EBITDA=%.1f, Coverage=%.1f → %.3f [%s]",
                          attrs.get("sp_rating"), debtEbitda, coverage, composite, band)
        );
    }

    private RiskScore.RiskBand classifyBand(double s) {
        if (s >= 0.70) return RiskScore.RiskBand.LOW;
        if (s >= 0.50) return RiskScore.RiskBand.MEDIUM;
        if (s >= 0.30) return RiskScore.RiskBand.HIGH;
        return RiskScore.RiskBand.CRITICAL;
    }

    private double toDouble(Object v, double d) {
        try { return v != null ? Double.parseDouble(v.toString()) : d; }
        catch (NumberFormatException e) { return d; }
    }
}

// --- Regulatory Reporter (IFRS 9 + Basel III IRB for corporate) ---
public class CorporateRegulatoryReportGenerator implements RegulatoryReportGenerator {

    @Override
    public RegulatoryReport generate(CreditFile creditFile, RiskScore riskScore) {
        Map<String, Object> sections = new LinkedHashMap<>();

        // Basel III IRB Advanced approach for corporate
        double pd  = estimatePD(riskScore.score());
        double lgd = 0.40;   // corporate senior secured standard
        double ead = creditFile.getRequestedAmount();
```

```
        sections.put("approach",             "IRB_ADVANCED");
        sections.put("pd",                   pd);
        sections.put("lgd",                  lgd);
        sections.put("ead",                  ead);
        sections.put("maturity_years",       3.0);
        sections.put("rwa",                  computeCorpRWA(pd, lgd, ead));
        sections.put("capital_requirement",  computeCorpRWA(pd, lgd, ead) * 0.08);

        // IFRS 9
        sections.put("ifrs9_stage",          resolveStage(riskScore));
        sections.put("ecl_12m",              pd * lgd * ead * 0.12);
        sections.put("ecl_lifetime",         pd * lgd * ead);

        // Large Exposure (CRR Article 395 — >10% of eligible capital)
        sections.put("large_exposure_flag",  ead > 50_000_000);

        return new RegulatoryReport(
            "REG-CORP-" + creditFile.getFileId(),
            "IFRS9 / Basel III IRB / CRR2",
            creditFile.getFileId(), sections
        );
    }

    private double estimatePD(double score) { return Math.max(0.0005, (1.0 - score) * 0.30); }

    private double computeCorpRWA(double pd, double lgd, double ead) {
        // Simplified Basel III corporate IRB formula
        double correlation = 0.12 * (1 - Math.exp(-50 * pd)) / (1 - Math.exp(-50))
                           + 0.24 * (1 - (1 - Math.exp(-50 * pd)) / (1 - Math.exp(-50)));
        double matAdj = (1 + (3.0 - 2.5) * 0.05) / (1 - 1.5 * 0.05);
        double capitalReq = (lgd * normalCDF(inverseCDF(pd) / Math.sqrt(1 - correlation) + Math.sqrt(correlation
        return Math.max(0, capitalReq * ead * 12.5);
    }

    private int resolveStage(RiskScore rs) {
        return switch (rs.band()) { case LOW -> 1; case MEDIUM -> 2; default -> 3; };
    }

    // Approximations — use Apache Commons Math in production
    private double normalCDF(double x)    { return 0.5 * (1 + erf(x / Math.sqrt(2))); }
    private double erf(double x)          { return Math.signum(x) * Math.sqrt(1 - Math.exp(-x * x * (4.0 / Math.P
    private double inverseCDF(double p)   { return Math.sqrt(2) * inverseErf(2 * p - 1); }
    private double inverseErf(double x)   { double a = 0.147; double ln = Math.log(1 - x * x); double t = 2 / (Ma
}

// --- Approval Handler ---
public class CorporateApprovalWorkflowHandler implements ApprovalWorkflowHandler {

    @Override
    public ApprovalDecision handle(CreditFile creditFile, RiskScore riskScore) {
        double amount = creditFile.getRequestedAmount();
        double score  = riskScore.score();
```

```
        if (score < 0.30) {
            return new ApprovalDecision(creditFile.getFileId(), false,
                "Corporate credit declined: critical risk band", "CREDIT_RISK_COMMITTEE");
        }

        // Tiered approval by amount and risk
        String tier = determineTier(score, amount);
        boolean approved = score >= 0.50 || (score >= 0.35 && amount < 5_000_000);

        return new ApprovalDecision(creditFile.getFileId(), approved,
            approved ? "Approved via " + tier : "Declined — escalated to credit committee", tier);
    }

    private String determineTier(double score, double amount) {
        if (amount > 100_000_000) return "BOARD_CREDIT_COMMITTEE";
        if (amount > 25_000_000)  return "SENIOR_CREDIT_COMMITTEE";
        if (amount > 5_000_000)   return "CORPORATE_CREDIT_MANAGER";
        if (score >= 0.70)        return "CORPORATE_UNDERWRITER_L2";
        return "CORPORATE_UNDERWRITER_L1";
    }
}

// --- Factory ---
public class CorporateCreditProcessingFactory implements CreditProcessingFactory {

    @Override public CreditFileParser          createParser()                   { return new CorporateCreditFileP
    @Override public RiskScoringEngine         createRiskScoringEngine()        { return new CorporateRiskScoring
    @Override public RegulatoryReportGenerator createRegulatoryReportGenerator() { return new CorporateRegulatory
    @Override public ApprovalWorkflowHandler   createApprovalWorkflowHandler()  { return new CorporateApprovalWor
    @Override public CreditType                supportedType()                  { return CreditType.CORPORATE; }
}
```

---

6. Mortgage Credit Family

```
// --- Parser ---
public class MortgageCreditFileParser implements CreditFileParser {

    @Override
    public CreditFile parse(byte[] rawData) {
        String json = new String(rawData, StandardCharsets.UTF_8);
        Map<String, Object> attrs = new HashMap<>();
        attrs.put("ltv_ratio",        extractField(json, "ltv_ratio"));        // Loan-to-Value
        attrs.put("property_value",   extractField(json, "property_value"));
        attrs.put("fico_score",       extractField(json, "fico_score"));
        attrs.put("dti_ratio",        extractField(json, "dti_ratio"));
        attrs.put("property_type",    extractField(json, "property_type"));    // PRIMARY, INVESTMENT
        attrs.put("loan_term_years",  extractField(json, "loan_term_years"));
```

```
        attrs.put("is_fixed_rate",    extractField(json, "is_fixed_rate"));

        return new CreditFile(
            UUID.randomUUID().toString(),
            extractField(json, "applicant_id").toString(),
            CreditType.MORTGAGE,
            Double.parseDouble(extractField(json, "loan_amount").toString()),
            attrs
        );
    }

    private Object extractField(String json, String field) { return "[" + field + "_value]"; }
}

// --- Risk Scorer ---
public class MortgageRiskScoringEngine implements RiskScoringEngine {

    @Override
    public RiskScore score(CreditFile creditFile) {
        Map<String, Object> attrs = creditFile.getAttributes();

        double ltv   = toDouble(attrs.get("ltv_ratio"),  0.80);   // lower LTV = safer
        double fico  = toDouble(attrs.get("fico_score"),  700.0);
        double dti   = toDouble(attrs.get("dti_ratio"),   0.43);

        double ltvScore  = Math.max(0, 1.0 - (ltv / 1.2));       // penalise >100% LTV
        double ficoScore = (fico - 300) / 550.0;
        double dtiScore  = 1.0 - Math.min(dti / 0.60, 1.0);

        // Property type adjustment
        boolean isInvestment = "INVESTMENT".equalsIgnoreCase(
            attrs.getOrDefault("property_type", "PRIMARY").toString());
        double propertyAdj = isInvestment ? 0.90 : 1.00;          // 10% haircut for BTL

        double composite = ((ltvScore  * 0.40)
                          +  (ficoScore * 0.35)
                          +  (dtiScore  * 0.25)) * propertyAdj;

        RiskScore.RiskBand band = classifyBand(composite);
        return new RiskScore(
            creditFile.getFileId(), composite, band,
            String.format("Mortgage scoring: LTV=%.2f, FICO=%.0f, DTI=%.2f, %s → %.3f [%s]",
                          ltv, fico, dti, isInvestment ? "BTL" : "OO", composite, band)
        );
    }

    private RiskScore.RiskBand classifyBand(double s) {
        if (s >= 0.70) return RiskScore.RiskBand.LOW;
        if (s >= 0.50) return RiskScore.RiskBand.MEDIUM;
        if (s >= 0.30) return RiskScore.RiskBand.HIGH;
        return RiskScore.RiskBand.CRITICAL;
    }
```

```
    private double toDouble(Object v, double d) {
        try { return v != null ? Double.parseDouble(v.toString()) : d; }
        catch (NumberFormatException e) { return d; }
    }
}

// --- Regulatory Reporter (IFRS 9 + Basel III + stress test for mortgage) ---
public class MortgageRegulatoryReportGenerator implements RegulatoryReportGenerator {

    @Override
    public RegulatoryReport generate(CreditFile creditFile, RiskScore riskScore) {
        Map<String, Object> attrs = creditFile.getAttributes();
        double ltv    = toDouble(attrs.get("ltv_ratio"), 0.80);
        double loanAmt = creditFile.getRequestedAmount();

        Map<String, Object> sections = new LinkedHashMap<>();

        // Basel III mortgage risk weights (standardised approach CRR2 Article 124)
        double riskWeight = resolveRegulatoryRiskWeight(ltv);
        sections.put("risk_weight",          riskWeight);
        sections.put("rwa",                  loanAmt * riskWeight);
        sections.put("capital_requirement",  loanAmt * riskWeight * 0.08);

        // IFRS 9
        double pd  = estimatePD(riskScore.score());
        double lgd = deriveLGD(ltv);       // secured — LTV-driven LGD
        sections.put("ifrs9_stage", resolveStage(riskScore));
        sections.put("pd",  pd);
        sections.put("lgd", lgd);
        sections.put("ecl", pd * lgd * loanAmt);

        // Stress test (EBA/PRA mortgage stress)
        sections.put("stressed_ltv_at_minus20pct_hpi", ltv / 0.80);
        sections.put("stressed_rwa",  loanAmt * resolveRegulatoryRiskWeight(ltv / 0.80) * 1.25);

        return new RegulatoryReport(
            "REG-MTG-" + creditFile.getFileId(),
            "IFRS9 / Basel III / PRA SS13-16",
            creditFile.getFileId(), sections
        );
    }

    private double resolveRegulatoryRiskWeight(double ltv) {
        // CRR2 Article 124 lookup
        if (ltv <= 0.50) return 0.20;
        if (ltv <= 0.60) return 0.25;
        if (ltv <= 0.80) return 0.35;
        if (ltv <= 0.90) return 0.50;
        if (ltv <= 1.00) return 0.75;
        return 1.05;   // >100% LTV
    }

    private double deriveLGD(double ltv) {
```

```
        // Secured LGD: rises with LTV
        return Math.min(0.10 + (ltv * 0.40), 0.75);
    }

    private double estimatePD(double score) { return Math.max(0.002, (1 - score) * 0.25); }
    private int    resolveStage(RiskScore rs) { return switch (rs.band()) { case LOW -> 1; case MEDIUM -> 2; defa
    private double toDouble(Object v, double d) { try { return v != null ? Double.parseDouble(v.toString()) : d;
}

// --- Approval Handler ---
public class MortgageApprovalWorkflowHandler implements ApprovalWorkflowHandler {

    private static final double MAX_AUTO_APPROVE_LTV    = 0.75;
    private static final double MAX_AUTO_APPROVE_AMOUNT = 500_000;

    @Override
    public ApprovalDecision handle(CreditFile creditFile, RiskScore riskScore) {
        Map<String, Object> attrs = creditFile.getAttributes();
        double ltv    = toDouble(attrs.get("ltv_ratio"), 0.80);
        double amount = creditFile.getRequestedAmount();
        double score  = riskScore.score();

        // Hard declines
        if (ltv > 0.95) {
            return new ApprovalDecision(creditFile.getFileId(), false,
                "LTV exceeds 95% — above regulatory lending cap", "AUTOMATED_POLICY_CHECK");
        }
        if (score < 0.30) {
            return new ApprovalDecision(creditFile.getFileId(), false,
                "Mortgage declined: critical risk score", "MORTGAGE_CREDIT_COMMITTEE");
        }

        // Auto-approve: low risk, conservative LTV, within limit
        if (score >= 0.75 && ltv <= MAX_AUTO_APPROVE_LTV && amount <= MAX_AUTO_APPROVE_AMOUNT) {
            return new ApprovalDecision(creditFile.getFileId(), true,
                "Auto-approved: low risk mortgage within policy parameters",
                "AUTOMATED_UNDERWRITING_SYSTEM");
        }

        // Tiered manual underwriting
        String tier = amount > 1_000_000 ? "SENIOR_MORTGAGE_UNDERWRITER"
                    : amount > 500_000   ? "MORTGAGE_UNDERWRITER_L2"
                    :                      "MORTGAGE_UNDERWRITER_L1";

        return new ApprovalDecision(creditFile.getFileId(), score >= 0.50,
            score >= 0.50 ? "Approved — manual underwrite required" : "Declined — high risk mortgage",
            tier);
    }

    private double toDouble(Object v, double d) { try { return v != null ? Double.parseDouble(v.toString()) : d;
}

// --- Factory ---
```

```
public class MortgageCreditProcessingFactory implements CreditProcessingFactory {

    @Override public CreditFileParser          createParser()                   { return new MortgageCreditFilePa
    @Override public RiskScoringEngine         createRiskScoringEngine()        { return new MortgageRiskScoringE
    @Override public RegulatoryReportGenerator createRegulatoryReportGenerator() { return new MortgageRegulatoryR
    @Override public ApprovalWorkflowHandler   createApprovalWorkflowHandler()  { return new MortgageApprovalWork
    @Override public CreditType                supportedType()                  { return CreditType.MORTGAGE; }
}
```

---

7. Factory Registry & Orchestrator

```
/**
 * Registry — decouples client code from concrete factory classes.
 * New credit types register themselves without changing existing code.
 */
public class CreditProcessingFactoryRegistry {

    private static final Map<CreditType, CreditProcessingFactory> REGISTRY = new EnumMap<>(CreditType.class);

    static {
        register(new RetailCreditProcessingFactory());
        register(new CorporateCreditProcessingFactory());
        register(new MortgageCreditProcessingFactory());
    }

    public static void register(CreditProcessingFactory factory) {
        REGISTRY.put(factory.supportedType(), factory);
    }

    public static CreditProcessingFactory getFactory(CreditType type) {
        CreditProcessingFactory factory = REGISTRY.get(type);
        if (factory == null) {
            throw new IllegalArgumentException("No factory registered for credit type: " + type);
        }
        return factory;
    }
}

/**
 * Orchestrator — the only class that touches the abstract factory.
 * It never knows which concrete products it is working with.
 */
public class CreditFileProcessingOrchestrator {

    public ProcessingResult process(byte[] rawFile, CreditType creditType) {
        CreditProcessingFactory factory = CreditProcessingFactoryRegistry.getFactory(creditType);

        // Use the factory to obtain the full product family
```

```
        CreditFileParser          parser   = factory.createParser();
        RiskScoringEngine         scorer   = factory.createRiskScoringEngine();
        RegulatoryReportGenerator reporter = factory.createRegulatoryReportGenerator();
        ApprovalWorkflowHandler   approver = factory.createApprovalWorkflowHandler();

        // Pipeline: parse → score → report → decide
        CreditFile         creditFile  = parser.parse(rawFile);
        RiskScore          riskScore   = scorer.score(creditFile);
        RegulatoryReport   report      = reporter.generate(creditFile, riskScore);
        ApprovalDecision   decision    = approver.handle(creditFile, riskScore);

        return new ProcessingResult(creditFile, riskScore, report, decision);
    }

    public record ProcessingResult(
        CreditFile       creditFile,
        RiskScore        riskScore,
        RegulatoryReport regulatoryReport,
        ApprovalDecision decision
    ) {}
}
```

---

8. Client / Main

```
public class CreditProcessingApplication {

    public static void main(String[] args) {
        CreditFileProcessingOrchestrator orchestrator = new CreditFileProcessingOrchestrator();

        // Simulate three incoming credit file payloads
        byte[] retailPayload    = loadPayload("retail-file.json");
        byte[] corporatePayload = loadPayload("corporate-file.json");
        byte[] mortgagePayload  = loadPayload("mortgage-file.json");

        process(orchestrator, retailPayload,    CreditType.RETAIL);
        process(orchestrator, corporatePayload, CreditType.CORPORATE);
        process(orchestrator, mortgagePayload,  CreditType.MORTGAGE);
    }

    private static void process(CreditFileProcessingOrchestrator orchestrator,
                                byte[] payload, CreditType type) {
        System.out.printf("%n=== Processing %s Credit File ===%n", type);

        CreditFileProcessingOrchestrator.ProcessingResult result =
            orchestrator.process(payload, type);

        System.out.println("Risk Score   : " + result.riskScore().score()
                           + " [" + result.riskScore().band() + "]");
```

```
        System.out.println("Rationale    : " + result.riskScore().rationale());
        System.out.println("Reg Report   : " + result.regulatoryReport().framework()
                           + " sections=" + result.regulatoryReport().sections().keySet());
        System.out.println("Decision     : " + (result.decision().approved() ? "APPROVED" : "DECLINED")
                           + " via " + result.decision().approverTier());
        System.out.println("Reason       : " + result.decision().reason());
    }

    private static byte[] loadPayload(String name) {
        // In production: load from message queue, file system, or API
        return ("{ \"applicant_id\": \"A001\", \"fico_score\": \"720\", "
              + "\"annual_income\": \"85000\", \"dti_ratio\": \"0.38\", "
              + "\"requested_amount\": \"35000\", \"employment_status\": \"EMPLOYED\" }").getBytes();
    }
}
```

Pattern Structure Summary

```
CreditProcessingFactory  (Abstract Factory)
├── RetailCreditProcessingFactory     → Retail product family
├── CorporateCreditProcessingFactory  → Corporate product family
└── MortgageCreditProcessingFactory   → Mortgage product family

Each factory creates:
  createParser()                → CreditFileParser
  createRiskScoringEngine()     → RiskScoringEngine
  createRegulatoryReportGenerator() → RegulatoryReportGenerator
  createApprovalWorkflowHandler() → ApprovalWorkflowHandler

CreditProcessingFactoryRegistry  → resolves factory by CreditType enum
CreditFileProcessingOrchestrator → runs the 4-step pipeline using only interfaces
```

Key Design Decisions

Decision	Rationale
`CreditType` enum as registry key	Type-safe dispatch; no stringly-typed switching
Registry pattern wrapping factories	Open/Closed — new credit types added by registering, not modifying
`record` for value objects	Immutable domain outputs (`RiskScore` , `ApprovalDecision` , etc.)

Decision	Rationale
Orchestrator depends only on interfaces	Client code is fully isolated from all 12 concrete product classes
Regulatory math per product family	Basel III IRB vs standardised weights differ by credit type — factories enforce correct pairing
LTV-driven LGD in mortgage	Reflects actual secured-credit recovery rates, not a flat assumption